Diversio HRIS Import Preview

High Level Architecture and Project Knowledge Base

Purpose

This document is the detailed technical reference for the HRIS Import Preview project. It explains what the application does, how data moves through the system, what each module is responsible for, how validation and hierarchy rules work, how reporting cycles are detected, how Django connects the pieces, and how the project is tested.

1. Application Overview

The application accepts an HRIS employee CSV through a web interface and analyzes it before import. It identifies data quality problems, resolves valid manager relationships, determines the reporting hierarchy, detects reporting cycles, and presents the findings as an import preview.

The application is intentionally focused. It does not act as a complete HR management system and does not persist employee hierarchy data in a database.

2. High Level Processing Flow

```
Browser → Django upload view → Import service → CSV parser → Normalizer → Identity validator → Manager resolver → Hierarchy analyzer → Cycle detector → AnalysisResult → Results page
```

Each stage has a single responsibility. The core processing stages are independent of Django, which keeps the business logic easier to test and maintain.

3. Repository Structure

The repository is organized by responsibility rather than placing all logic in the Django views. The following table describes the important project files.

Path	Responsibility
config/	Django project configuration and URL entry point.
hris/domain/	Core domain objects, result structures, and domain errors.
hris/importers/	CSV parsing, normalization, and identity validation.
hris/hierarchy/	Manager resolution, hierarchy analysis, and cycle detection.
hris/services/	Coordinates the complete analysis pipeline.
hris/templates/hris/	Upload and analysis result pages.
hris/templatetags/	Template helper functions used by the UI.
hris/tests/	Automated tests for the application layers.
hris/forms.py	Validates the uploaded file at the Django form boundary.
hris/views.py	Handles HTTP requests and passes results to templates.
hris/urls.py	Application URL routes.
sample_data/sample_hris.csv	Sample HRIS input used for demonstration and verification.
manage.py	Django command line entry point.
requirements.txt	Python dependencies required to run the project.

Path	Responsibility
README.md	Setup, usage, architecture, and project documentation.

4. Domain Layer

The domain layer contains the objects exchanged by the processing stages. It has no dependency on Django or a database.

Employee represents a normalized employee record. It contains the employee identity, contact information, manager references, and department value used by the import model.

ValidationError stores the source row number and a human readable validation message so the UI can point the user back to the original CSV.

ManagerSummary stores a manager employee ID, name, and direct report count for presentation.

AnalysisResult is the final aggregate returned by the processing pipeline. It contains the source row count, accepted employees, validation errors, roots, manager summaries, and cyclic employees.

DomainError is the base domain exception. CSVStructureError represents structural CSV failures.

5. CSV Contract

The application is designed for the HRIS CSV contract used by the assessment. The required headers are:

```
employee_id, employee_name, email, manager_id, manager_email, department
```

Header order does not matter. CSV quoting is supported, so a name such as Alvarez, Renée can be represented correctly when quoted by the CSV file.

6. CSV Parsing

The parser is responsible for reading the CSV structure and producing source rows. It uses Python's standard CSV facilities and supports UTF 8 input with a UTF 8 byte order mark.

• Required headers are checked before row processing.

• Missing required headers result in a structural CSV error.

• Source row numbers are preserved so validation messages can identify the original row.

• Quoted fields containing commas are handled as a single CSV field.

• Raw field values are passed to the normalizer rather than being modified inside the parser.

7. Normalization

Normalization creates consistent values before identity and relationship checks are performed.

• Surrounding whitespace is removed from fields.

• Email and manager_email are converted to lowercase.

• employee_id and manager_id are trimmed but keep their original case.

• Empty normalized values are treated as missing.

• Internal whitespace in names and department values is preserved.

8. Identity Validation

Identity validation determines which source rows are safe to include in hierarchy analysis. employee_id and email are required. Employee IDs are case sensitive, while emails are compared after normalization.

Duplicate employee IDs and duplicate emails invalidate all rows sharing the duplicated identity. Invalid identity rows are excluded from accepted_employees and are therefore not available for manager

lookup.

The implementation uses dictionaries and sets for duplicate detection, giving expected O(n) processing rather than comparing every row against every other row.

9. Manager Resolution

Manager resolution runs after identity validation. The resolver builds employee indexes by ID and email, allowing manager references to be resolved without scanning the full employee collection for every row.

- No manager ID and no manager email means the employee is a genuine root candidate.
- manager_id is resolved using a case sensitive employee ID lookup.
- manager_email is resolved using normalized lowercase email.
- When both references are supplied, both must identify the same employee.
- A missing manager produces an error and no reporting relationship.
- A conflicting manager ID and manager email produce an error and no reporting relationship.
- Self management produces an error and no reporting relationship.
- An employee with a manager error remains identity accepted but is not treated as a root.

10. Hierarchy Analysis

Hierarchy analysis uses only valid reporting relationships returned by the manager resolver. Root employees come from the resolver's explicit no manager set. Direct report counts are calculated from valid employee to manager relationships.

A manager summary is created only when an employee has at least one valid direct report. Results follow accepted employee source order where deterministic ordering is required.

11. Reporting Cycle Detection

Because each employee can have at most one manager, the valid reporting structure can be represented as a directed functional graph. The cycle detector uses iterative depth first traversal with three states.

```
WHITE means unvisited. GRAY means currently being explored. BLACK means fully processed.
```

- Start with an unvisited employee that has a valid manager relationship.
- Follow the manager chain and mark active nodes GRAY.
- If the next node is WHITE, continue the traversal.
- If the next node is GRAY, a cycle has been reached.
- Only the repeated node and the remainder of the active path form the cycle.
- Employees that only lead into the cycle are not reported as cycle members.
- The traversal is iterative, so long reporting chains do not depend on Python recursion depth.

Example

```
D → B → C → B
```

The cycle consists of B and C. D is not cyclic because D only leads into the cycle.

12. Django Integration

Django provides the web boundary. The upload form validates the incoming file, the view handles the request, and the import service coordinates the framework independent processing stages.

The service returns a single AnalysisResult to the view. The result template is responsible for presenting the already calculated information rather than implementing hierarchy logic.

13. User Interface

The upload page provides a focused file selection experience and explains that the application expects an HRIS CSV. The results page presents summary counts followed by validation errors, root employees, managers and direct reports, and reporting cycles.

The current interface is server rendered. Styling is contained in the templates, so a separate CSS package is not required by the current implementation.

14. End to End Request Flow

1. The user opens the upload page.
2. The user selects an HRIS CSV file.
3. Django receives the uploaded file.
4. The form validates the upload boundary.
5. The import service starts the analysis.
6. The parser reads the source rows.
7. The normalizer standardizes field values.
8. Identity validation determines accepted employees.
9. Manager resolution creates valid reporting relationships.

10. Hierarchy analysis calculates roots and direct report counts.
11. Cycle detection identifies actual cycle members.
12. AnalysisResult is assembled.
13. Django renders the result page.

15. Testing Strategy

The test suite is organized around the processing stages and the edge cases that can change the final hierarchy result.

Test area	Coverage
CSV parser	Quoted values, BOM, header order, missing headers, invalid encoding, and row numbers.
Normalizer	Whitespace, email casing, ID case preservation, and empty values.
Validation	Required fields, duplicate IDs, duplicate emails, and invalid row exclusion.
Manager resolution	ID lookup, email lookup, missing managers, conflicts, self management, and root semantics.
Hierarchy	Root detection, direct report counts, ordering, and invalid relationship exclusion.
Cycle detection	Simple cycles, incoming chains, multiple cycles, self loops, and non cycle followers.
Forms and service	Upload validation and complete pipeline orchestration.

16. Sample Data

The supplied sample HRIS file contains 25 employee data rows. The completed pipeline produces 25 identity accepted employees, 22 valid reporting relationships, 2 manager resolution errors, and 1 genuine root.

The two manager errors are a missing manager reference for Casey and conflicting manager ID and manager email references for Riley. Neither employee is classified as a root.

The sample also contains a three employee cycle involving DIV 1701, DIV 1702, and DIV 1703.

17. Performance Characteristics

Stage	Expected complexity
Parsing	O(n)
Normalization	O(n)
Identity validation	O(n) expected
Manager resolution	O(n) expected
Hierarchy analysis	O(n)
Cycle detection	O(n)
Overall memory	O(n)

18. Important Data Distinctions

• Source rows and accepted employees are different. Identity invalid rows remain source rows but are not accepted employees.

• Identity errors and manager errors are different. A manager error does not invalidate an otherwise valid employee identity.

• A missing manager is not a root. Root status requires both manager references to be blank.

• A cycle follower is not a cycle member. Only employees inside the repeated graph segment are reported as cyclic.

• A valid cycle relationship still counts as a valid reporting relationship for direct report analysis.

19. Scope and Trade Offs

• No database persistence is required for the assessment workflow.

• No authentication or user account system is included because it is outside the assessment scope.

• The application follows the supplied HRIS CSV contract rather than acting as a general CSV analytics platform.

• The web layer remains server rendered to keep the project small and understandable.

• Further production work would focus on security controls, upload limits, deployment settings, broader malformed input coverage, and additional integration testing.

20. Local Development

Create and activate a virtual environment, install the dependencies, run Django checks, run the tests, and start the development server.

```
python -m venv .venv
pip install -r requirements.txt
python manage.py check
python manage.py test
python manage.py runserver
```

The application can then be opened at http://127.0.0.1:8000/ and the sample file can be uploaded.

21. Final Project Explanation

In practical terms, this application answers one main question: what would happen if this HRIS employee file were imported? It first determines which employee records are trustworthy, then determines who reports to whom, then identifies the top level employees, counts direct reports, and checks whether the reporting structure contains cycles. The result is shown before any persistent import takes place.

The architecture keeps parsing, validation, relationship resolution, hierarchy analysis, cycle detection, and presentation separate. That makes the system easier to understand, test, explain in an assessment, and extend later.